YOLOv3: An Incremental Improvement

Joseph Redmon Ali Farhadi
University of Washington

Abstract

We present some updates to YOLO! We made a bunch of little design changes to make it better. We also trained this new network that's pretty swell. It's a little bigger than last time but more accurate. It's still fast though, don't worry. At 320×320 *YOLOv3 runs in 22 ms at 28.2 mAP, as accurate as SSD but three times faster. When we look at the old .5 IOU mAP detection metric YOLOv3 is quite good. It achieves* 57.9 AP_{50} *in 51 ms on a Titan X, compared to* 57.5 AP_{50} *in 198 ms by RetinaNet, similar performance but 3.8*$\times$ *faster. As always, all the code is online at* https://pjreddie.com/yolo/.

1. Introduction

Sometimes you just kinda phone it in for a year, you know? I didn't do a whole lot of research this year. Spent a lot of time on Twitter. Played around with GANs a little. I had a little momentum left over from last year [12] [1]; I managed to make some improvements to YOLO. But, honestly, nothing like super interesting, just a bunch of small changes that make it better. I also helped out with other people's research a little.

Actually, that's what brings us here today. We have a camera-ready deadline [4] and we need to cite some of the random updates I made to YOLO but we don't have a source. So get ready for a TECH REPORT!

The great thing about tech reports is that they don't need intros, y'all know why we're here. So the end of this introduction will signpost for the rest of the paper. First we'll tell you what the deal is with YOLOv3. Then we'll tell you how we do. We'll also tell you about some things we tried that didn't work. Finally we'll contemplate what this all means.

2. The Deal

So here's the deal with YOLOv3: We mostly took good ideas from other people. We also trained a new classifier network that's better than the other ones. We'll just take you through the whole system from scratch so you can understand it all.

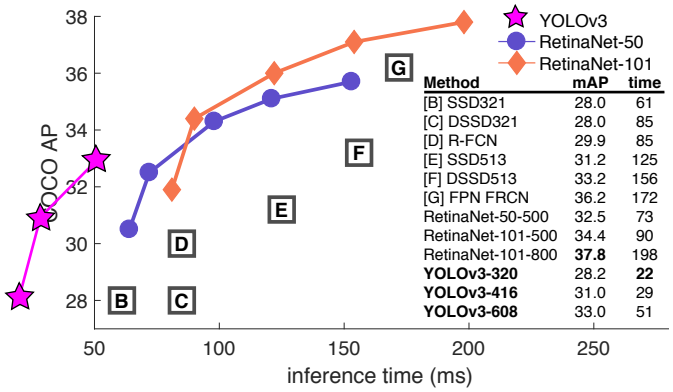

Figure 1. We adapt this figure from the Focal Loss paper [9]. YOLOv3 runs significantly faster than other detection methods with comparable performance. Times from either an M40 or Titan X, they are basically the same GPU.

2.1. Bounding Box Prediction

Following YOLO9000 our system predicts bounding boxes using dimension clusters as anchor boxes [15]. The network predicts 4 coordinates for each bounding box, t_x, t_y, t_w, t_h. If the cell is offset from the top left corner of the image by (c_x, c_y) and the bounding box prior has width and height p_w, p_h, then the predictions correspond to:

$$
\begin{aligned}
b_x &= \sigma(t_x) + c_x \\
b_y &= \sigma(t_y) + c_y \\
b_w &= p_w e^{t_w} \\
b_h &= p_h e^{t_h}
\end{aligned}
$$

During training we use sum of squared error loss. If the ground truth for some coordinate prediction is $\hat{t}_*$ our gradient is the ground truth value (computed from the ground truth box) minus our prediction: $\hat{t}_* - t_*$. This ground truth value can be easily computed by inverting the equations above.

YOLOv3 predicts an objectness score for each bounding box using logistic regression. This should be 1 if the bounding box prior overlaps a ground truth object by more than any other bounding box prior. If the bounding box prior

YOLOv3：一种渐进式的改进

约瑟夫·雷德蒙 阿里·法哈迪
华盛顿大学

摘要

We present some updates to YOLO! We made a bunch of little design changes to make it better. We also trained this new network that's pretty swell. It's a little bigger than last time but more accurate. It's still fast though, don't worry. At 320 × 320 *YOLOv3 runs in 22 ms at 28.2 mAP, as accurate as SSD but three times faster. When we look at the old .5 IOU mAP detection metric YOLOv3 is quite good. It achieves* 57.9 AP_{50} *in 51 ms on a Titan X, compared to* 57.5 AP_{50} *in 198 ms by RetinaNet, similar performance but 3.8× faster. As always, all the code is online at* https://pjreddie.com/yolo/.

1. 引言

有时候你就是会浑浑噩噩混上一年，懂吧？今年我没做太多研究。花了很多时间刷推特。稍微摆弄了一下GAN。我靠着去年留下的一点余势[12][1]，总算给YOLO做了一些改进。但说实话，没什么特别有意思的，就是一堆让它更好用的小改动。我也稍微帮别人做了点研究。

实际上，这正是我们今天聚集于此的原因。我们面临一个定稿截止日期[4]，需要引用我对YOLO所做的一些随机更新，却没有现成的文献来源。所以，准备好迎接一份技术报告吧！

技术报告的好处在于无需引言，各位都清楚我们为何在此。因此，这段引言的结尾将为全文提供指引。首先，我们将说明YOLOv3的核心内容。接着，我们会介绍我们的实现方法。同时也会分享一些我们尝试过但未成功的方案。最后，我们将探讨这一切的意义所在。

2. 协议

所以关于YOLOv3的情况是这样的：我们主要借鉴了他人的优秀思路。我们还训练了一个新的分类网络，它比其他网络表现更佳。接下来我们将从头开始带你了解整个系统，以便你能完全理解它。

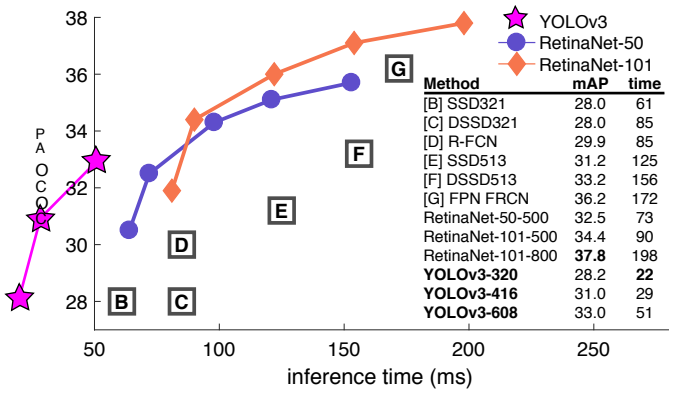

图1. 我们根据Focal Loss论文[9]改编此图。YOLOv3在性能相当的情况下，运行速度显著快于其他检测方法。时间数据基于M40或Titan X显卡，两者本质上是相同的GPU架构。

2.1. 边界框预测

继YOLO9000之后，我们的系统使用维度聚类得到的锚框来预测边界框[15]。网络为每个边界框预测4个坐标，t_x、t_y、t_w、t_h。若单元格从图像左上角偏移(c_x, c_y)，且边界框先验的宽度和高度为p_w、p_h，则预测值对应为：

$$
\begin{aligned}
b_x &= \sigma(t_x) + c_x \\
b_y &= \sigma(t_y) + c_y \\
b_w &= p_w e^{t_w} \\
b_h &= p_h e^{t_h}
\end{aligned}
$$

在训练过程中，我们使用平方误差损失之和。如果某个坐标预测的真实标签是 $\hat{t}_*$，我们的梯度就是真实值（根据真实框计算得出）减去预测值：$\hat{t}_* - t_*$。通过反转上述方程，可以轻松计算出这个真实值。

YOLOv3使用逻辑回归为每个边界框预测一个目标性分数。如果边界框先验与真实目标的重叠度超过任何其他边界框先验，则该分数应为1。如果边界框先验

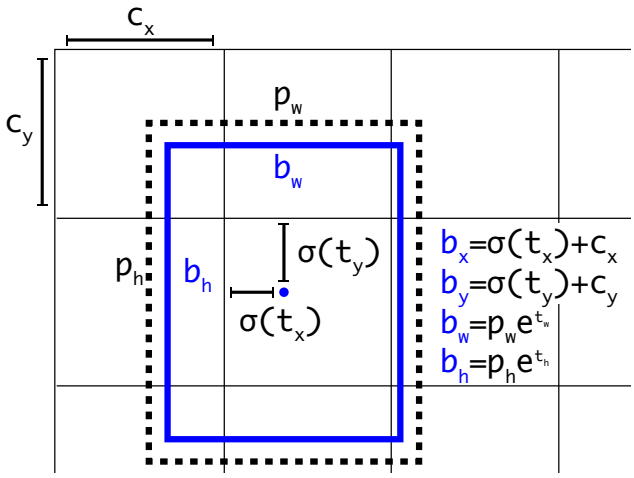

Figure 2. **Bounding boxes with dimension priors and location prediction.** We predict the width and height of the box as offsets from cluster centroids. We predict the center coordinates of the box relative to the location of filter application using a sigmoid function. This figure blatantly self-plagiarized from [15].

is not the best but does overlap a ground truth object by more than some threshold we ignore the prediction, following [17]. We use the threshold of .5. Unlike [17] our system only assigns one bounding box prior for each ground truth object. If a bounding box prior is not assigned to a ground truth object it incurs no loss for coordinate or class predictions, only objectness.

2.2. Class Prediction

Each box predicts the classes the bounding box may contain using multilabel classification. We do not use a softmax as we have found it is unnecessary for good performance, instead we simply use independent logistic classifiers. During training we use binary cross-entropy loss for the class predictions.

This formulation helps when we move to more complex domains like the Open Images Dataset [7]. In this dataset there are many overlapping labels (i.e. Woman and Person). Using a softmax imposes the assumption that each box has exactly one class which is often not the case. A multilabel approach better models the data.

2.3. Predictions Across Scales

YOLOv3 predicts boxes at 3 different scales. Our system extracts features from those scales using a similar concept to feature pyramid networks [8]. From our base feature extractor we add several convolutional layers. The last of these predicts a 3-d tensor encoding bounding box, objectness, and class predictions. In our experiments with COCO [10] we predict 3 boxes at each scale so the tensor is $N \times N \times [3 * (4 + 1 + 80)]$ for the 4 bounding box offsets, 1 objectness prediction, and 80 class predictions.

Next we take the feature map from 2 layers previous and upsample it by $2\times$. We also take a feature map from earlier in the network and merge it with our upsampled features using concatenation. This method allows us to get more meaningful semantic information from the upsampled features and finer-grained information from the earlier feature map. We then add a few more convolutional layers to process this combined feature map, and eventually predict a similar tensor, although now twice the size.

We perform the same design one more time to predict boxes for the final scale. Thus our predictions for the 3rd scale benefit from all the prior computation as well as fine-grained features from early on in the network.

We still use k-means clustering to determine our bounding box priors. We just sort of chose 9 clusters and 3 scales arbitrarily and then divide up the clusters evenly across scales. On the COCO dataset the 9 clusters were: $(10 \times 13), (16 \times 30), (33 \times 23), (30 \times 61), (62 \times 45), (59 \times 119), (116 \times 90), (156 \times 198), (373 \times 326)$.

2.4. Feature Extractor

We use a new network for performing feature extraction. Our new network is a hybrid approach between the network used in YOLOv2, Darknet-19, and that newfangled residual network stuff. Our network uses successive 3×3 and 1×1 convolutional layers but now has some shortcut connections as well and is significantly larger. It has 53 convolutional layers so we call it.... wait for it..... Darknet-53!

	Type	Filters	Size	Output
	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
1×	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
2×	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
8×	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
8×	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
4×	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

Table 1. **Darknet-53.**

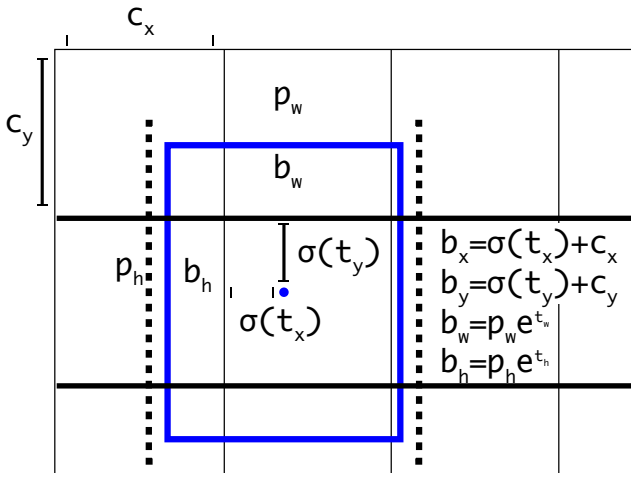

图2. 带有维度先验和位置预测的边界框。我们将边界框的宽度和高度预测为聚类中心点的偏移量，并使用sigmoid函数预测边界框中心坐标相对于滤波器应用位置的值。本图明显自我抄袭自[15]。

虽然不是最佳，但与某个真实对象的重叠度超过一定阈值时，我们会忽略该预测，遵循[17]的做法。我们使用的阈值为.5。与[17]不同，我们的系统仅为每个真实对象分配一个先验边界框。若某个先验边界框未被分配给真实对象，则其不会产生坐标或类别预测的损失，仅影响对象性判断。

2.2. 类别预测

每个边界框通过多标签分类预测其可能包含的类别。我们未使用softmax，因为发现其对良好性能并非必需，而是直接采用独立的逻辑分类器。在训练过程中，我们使用二元交叉熵损失函数进行类别预测。

当我们转向更复杂的领域，如开放图像数据集[7]时，这种表述方式会有所帮助。在该数据集中存在许多重叠的标签（例如“女性”和“人”）。使用softmax函数会强加每个边界框仅有一个类别的假设，而这往往不符合实际情况。采用多标签方法能更好地对数据进行建模。

2.3. 跨尺度预测

YOLOv3在三个不同尺度上预测边界框。我们的系统采用与特征金字塔网络[8]类似的概念，从这些尺度提取特征。在基础特征提取器之上，我们添加了若干卷积层。最后一层输出一个三维张量，用于编码边界框、目标置信度和类别预测。在COCO数据集[10]的实验中，我们在每个尺度预测3个边界框，因此张量维度为$N \times N \times [3 * (4 + 1 + 80)]$，分别对应4个边界框偏移量、1个目标置信度预测和80个类别预测。

接下来，我们提取前两层的特征图，并将其上采样2倍×。同时，我们从网络更早的层级中提取另一张特征图，并通过拼接方式将其与上采样后的特征融合。这种方法使我们能够从上采样的特征中获得更具语义意义的信息，并从早期特征图中获取更精细的细节。随后，我们添加若干卷积层来处理这一融合后的特征图，最终预测出一个结构相似但尺寸扩大一倍的张量。

我们再次执行相同的设计来预测最终尺度的边界框。因此，我们对第三尺度的预测受益于所有先前的计算以及网络早期阶段的细粒度特征。

我们仍然使用k-means聚类来确定边界框先验。我们只是随意选择了9个聚类和3个尺度，然后将聚类均匀分配到各个尺度上。在COCO数据集上，这9个聚类分别为：$(10 \times 13), (16 \times 30), (33 \times 23), (30 \times 61), (62 \times 45), (59 \times 119), (116 \times 90), (156 \times 198), (373 \times 326)$。

2.4. 特征提取器

我们采用一种新的网络进行特征提取。我们的新网络是YOLOv2中使用的Darknet-19与新兴残差网络技术的混合方法。该网络沿用连续的3×3与1×1卷积层结构，但新增了快捷连接机制，且网络规模显著扩大。它共包含53个卷积层，因此我们将其命名为——敬请期待——Darknet-53！

	Type	Filters	Size	Output
	Convolutional	32	3 × 3	256 × 256
	Convolutional	64	3 × 3 / 2	128 × 128
1×	Convolutional	32	1 × 1	
	Convolutional	64	3 × 3	
	Residual			128 × 128
	Convolutional	128	3 × 3 / 2	64 × 64
2×	Convolutional	64	1 × 1	
	Convolutional	128	3 × 3	
	Residual			64 × 64
	Convolutional	256	3 × 3 / 2	32 × 32
8×	Convolutional	128	1 × 1	
	Convolutional	256	3 × 3	
	Residual			32 × 32
	Convolutional	512	3 × 3 / 2	16 × 16
8×	Convolutional	256	1 × 1	
	Convolutional	512	3 × 3	
	Residual			16 × 16
	Convolutional	1024	3 × 3 / 2	8 × 8
4×	Convolutional	512	1 × 1	
	Convolutional	1024	3 × 3	
	Residual			8 × 8
	Avgpool		Global	
	Connected		1000	
	Softmax			

表 1. Darknet-53。

This new network is much more powerful than Darknet-19 but still more efficient than ResNet-101 or ResNet-152. Here are some ImageNet results:

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [15]	74.1	91.8	7.29	1246	**171**
ResNet-101[5]	77.1	93.7	19.7	1039	53
ResNet-152 [5]	**77.6**	**93.8**	29.4	1090	37
Darknet-53	77.2	**93.8**	18.7	**1457**	78

Table 2. **Comparison of backbones.** Accuracy, billions of operations, billion floating point operations per second, and FPS for various networks.

Each network is trained with identical settings and tested at 256×256, single crop accuracy. Run times are measured on a Titan X at 256×256. Thus Darknet-53 performs on par with state-of-the-art classifiers but with fewer floating point operations and more speed. Darknet-53 is better than ResNet-101 and $1.5\times$ faster. Darknet-53 has similar performance to ResNet-152 and is $2\times$ faster.

Darknet-53 also achieves the highest measured floating point operations per second. This means the network structure better utilizes the GPU, making it more efficient to evaluate and thus faster. That's mostly because ResNets have just way too many layers and aren't very efficient.

2.5. Training

We still train on full images with no hard negative mining or any of that stuff. We use multi-scale training, lots of data augmentation, batch normalization, all the standard stuff. We use the Darknet neural network framework for training and testing [14].

3. How We Do

YOLOv3 is pretty good! See table 3. In terms of COCOs weird average mean AP metric it is on par with the SSD variants but is $3\times$ faster. It is still quite a bit behind other models like RetinaNet in this metric though.

However, when we look at the "old" detection metric of mAP at IOU$=.5$ (or AP_{50} in the chart) YOLOv3 is very strong. It is almost on par with RetinaNet and far above the SSD variants. This indicates that YOLOv3 is a very strong detector that excels at producing decent boxes for objects. However, performance drops significantly as the IOU threshold increases indicating YOLOv3 struggles to get the boxes perfectly aligned with the object.

In the past YOLO struggled with small objects. However, now we see a reversal in that trend. With the new multi-scale predictions we see YOLOv3 has relatively high AP_S performance. However, it has comparatively worse performance on medium and larger size objects. More investigation is needed to get to the bottom of this.

When we plot accuracy vs speed on the AP_{50} metric (see figure 5) we see YOLOv3 has significant benefits over other detection systems. Namely, it's faster and better.

4. Things We Tried That Didn't Work

We tried lots of stuff while we were working on YOLOv3. A lot of it didn't work. Here's the stuff we can remember.

Anchor box x, y offset predictions. We tried using the normal anchor box prediction mechanism where you predict the x, y offset as a multiple of the box width or height using a linear activation. We found this formulation decreased model stability and didn't work very well.

Linear x, y predictions instead of logistic. We tried using a linear activation to directly predict the x, y offset instead of the logistic activation. This led to a couple point drop in mAP.

Focal loss. We tried using focal loss. It dropped our mAP about 2 points. YOLOv3 may already be robust to the problem focal loss is trying to solve because it has separate objectness predictions and conditional class predictions. Thus for most examples there is no loss from the class predictions? Or something? We aren't totally sure.

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2
YOLOv3 608×608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

Table 3. I'm seriously just stealing all these tables from [9] they take soooo long to make from scratch. Ok, YOLOv3 is doing alright. Keep in mind that RetinaNet has like $3.8\times$ longer to process an image. YOLOv3 is much better than SSD variants and comparable to state-of-the-art models on the AP_{50} metric.

这个新网络比Darknet-19强大得多，但仍比ResNet-101或ResNet-152更高效。以下是一些ImageNet的结果：

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [15]	74.1	91.8	7.29	1246	**171**
ResNet-101[5]	77.1	93.7	19.7	1039	53
ResNet-152 [5]	**77.6**	**93.8**	29.4	1090	37
Darknet-53	77.2	**93.8**	18.7	**1457**	78

表2. 骨干网络对比。各网络的准确率、操作数十亿次、每秒浮点运算数十亿次以及帧率。

每个网络都采用相同的设置进行训练，并在256×256分辨率下以单次裁剪精度进行测试。运行时间是在Titan X显卡上以256×256分辨率测量的。因此，Darknet-53的性能与最先进的分类器相当，但浮点运算更少、速度更快。Darknet-53优于ResNet-101，且速度快1.5×倍。Darknet-53与ResNet-152性能相近，但速度快2×倍。

Darknet-53 还实现了每秒最高的浮点运算性能测量值。这意味着该网络结构能更好地利用 GPU，使其评估效率更高，从而速度更快。这主要是因为 ResNet 的层数过多且效率不高。

2.5. 训练

我们仍然在完整图像上进行训练，不使用困难负样本挖掘或任何此类技术。我们采用多尺度训练、大量数据增强、批量归一化等所有标准方法。训练和测试均使用Darknet神经网络框架[14]。

3. 我们的做法

YOLOv3表现相当出色！参见表3。就COCO数据集那套独特的平均精度均值（AP）评估指标而言，它与SSD系列变体性能相当，但速度快了3×倍。不过相比其他模型仍有明显差距。

不过，在此指标中，像RetinaNet这样的模型表现。

然而，当我们观察IOU阈值为= .5时的“传统”检测指标mAP（或图表中的AP_{50}），YOLOv3表现非常出色。它几乎与RetinaNet持平，并远超SSD的各类变体。这表明YOLOv3是一款强大的检测器，擅长为物体生成合适的边界框。但随着IOU阈值提高，其性能显著下降，说明YOLOv3在使边界框与物体精确对齐方面仍存在困难。

过去YOLO在小物体检测上表现不佳。然而，现在这一趋势出现了逆转。通过新的多尺度预测，我们看到YOLOv3在AP_S指标上表现出相对较高的性能。但它在中等和较大尺寸物体上的性能相对较差。需要进一步研究以查明根本原因。

当我们在AP_{50}指标上绘制精度与速度的关系图时（见图5），可以看到YOLOv3相比其他检测系统具有显著优势。具体而言，它更快且更优。

4. 我们尝试过但未奏效的方法

在开发YOLOv3的过程中，我们尝试了许多方法，其中大部分并未奏效。以下是我们还能记得的一些尝试。

锚框 x, y 偏移量预测。我们尝试使用常规的锚框预测机制，即通过线性激活函数将 x, y 偏移量预测为框宽度或高度的倍数。我们发现这种设定会降低模型稳定性，且效果不佳。

线性x, y预测替代逻辑回归。我们尝试使用线性激活函数直接预测x, y偏移量，而非逻辑激活函数。这导致mAP指标下降了若干点。

焦点损失。我们尝试使用了焦点损失。它使我们的mAP下降了约2个百分点。YOLOv3可能已经对焦点损失试图解决的问题具有鲁棒性，因为它具有独立的物体性预测和条件类别预测。因此对于大多数样本，类别预测没有损失？或者类似的原因？我们并不完全确定。

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

表3。我真的是直接从[9]里偷了所有这些表格，从头开始制作它们太费时间了。好吧，YOLOv3表现还行。要记住，RetinaNet处理一张图像的时间大约是YOLOv3的3.8×倍。YOLOv3比SSD的变体好得多，并且在AP_{50}指标上与最先进的模型相当。

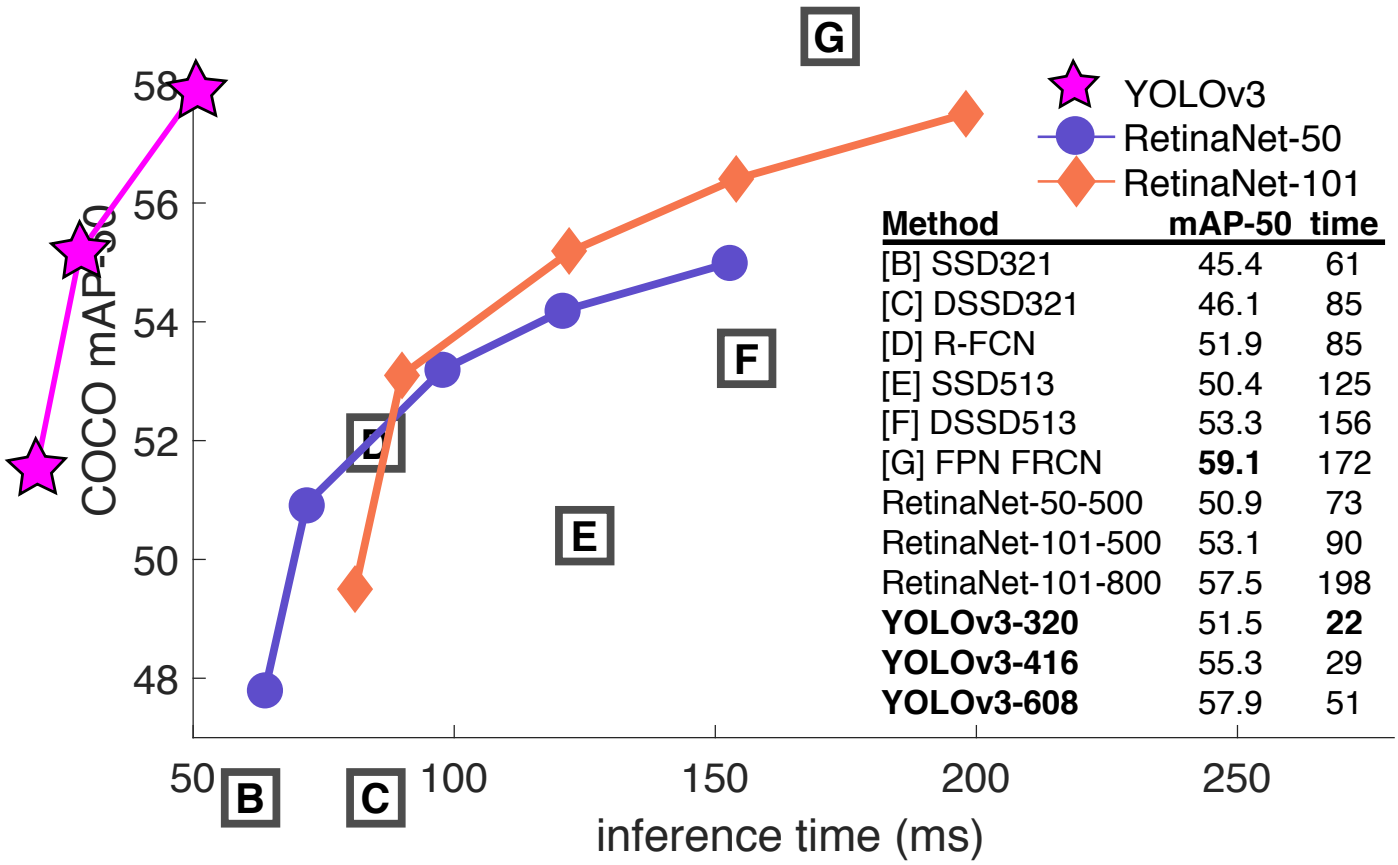

Figure 3. Again adapted from the [9], this time displaying speed/accuracy tradeoff on the mAP at .5 IOU metric. You can tell YOLOv3 is good because it's very high and far to the left. Can you cite your own paper? Guess who's going to try, this guy → [16]. Oh, I forgot, we also fix a data loading bug in YOLOv2, that helped by like 2 mAP. Just sneaking this in here to not throw off layout.

Dual IOU thresholds and truth assignment. Faster R-CNN uses two IOU thresholds during training. If a prediction overlaps the ground truth by .7 it is as a positive example, by $[.3 - .7]$ it is ignored, less than .3 for all ground truth objects it is a negative example. We tried a similar strategy but couldn't get good results.

We quite like our current formulation, it seems to be at a local optima at least. It is possible that some of these techniques could eventually produce good results, perhaps they just need some tuning to stabilize the training.

5. What This All Means

YOLOv3 is a good detector. It's fast, it's accurate. It's not as great on the COCO average AP between .5 and .95 IOU metric. But it's very good on the old detection metric of .5 IOU.

Why did we switch metrics anyway? The original COCO paper just has this cryptic sentence: "A full discussion of evaluation metrics will be added once the evaluation server is complete". Russakovsky et al report that that humans have a hard time distinguishing an IOU of .3 from .5! "Training humans to visually inspect a bounding box with IOU of 0.3 and distinguish it from one with IOU 0.5 is surprisingly difficult." [18] If humans have a hard time telling the difference, how much does it matter?

But maybe a better question is: "What are we going to do with these detectors now that we have them?" A lot of the people doing this research are at Google and Facebook. I guess at least we know the technology is in good hands and definitely won't be used to harvest your personal information and sell it to.... wait, you're saying that's exactly what it will be used for?? Oh.

Well the other people heavily funding vision research are the military and they've never done anything horrible like killing lots of people with new technology oh wait.....[1]

I have a lot of hope that most of the people using computer vision are just doing happy, good stuff with it, like counting the number of zebras in a national park [13], or tracking their cat as it wanders around their house [19]. But computer vision is already being put to questionable use and as researchers we have a responsibility to at least consider the harm our work might be doing and think of ways to mitigate it. We owe the world that much.

In closing, do not @ me. (Because I finally quit Twitter).

[1]The author is funded by the Office of Naval Research and Google.

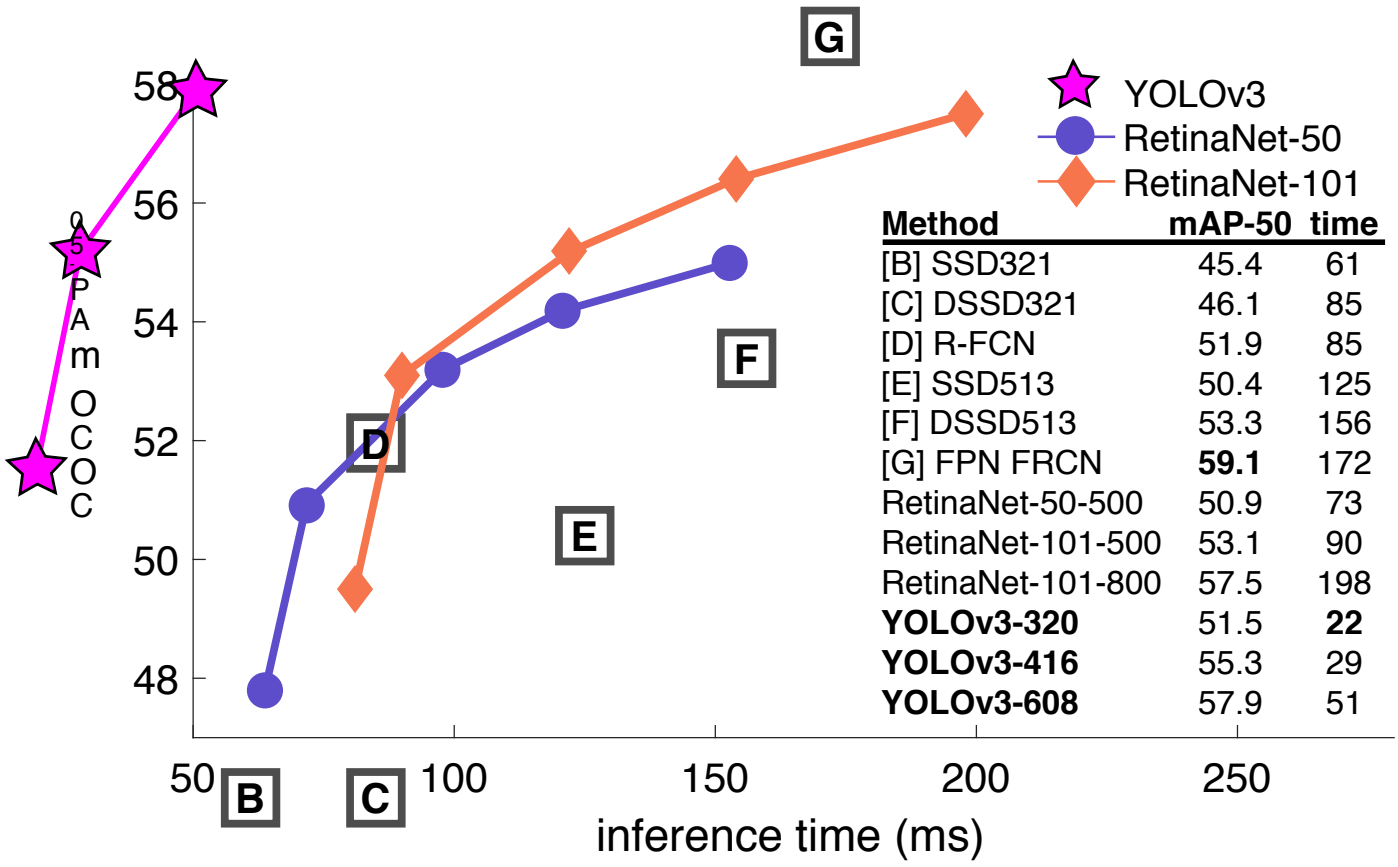

图3. 再次改编自[9]，这次展示的是在0.5 IOU指标下mAP的速度/精度权衡。你可以看出YOLOv3表现优异，因为它位置很高且非常靠左。能引用自己的论文吗？猜猜谁要试试——这位老兄→[16]。哦，我忘了，我们还修复了YOLOv2中的一个数据加载错误，这使mAP提升了约2个点。悄悄把这点加在这儿，以免打乱排版。

双重IOU阈值与真值分配。Faster R-CNN在训练过程中使用了两个IOU阈值。若预测框与真实框的重叠度达到0.7则视为正样本，在[.3 – .7]区间内则被忽略，若与所有真实目标的重叠度均低于0.3则视为负样本。我们尝试了类似策略但未能取得理想结果。

我们相当喜欢我们目前的表述，它至少看起来处于一个局部最优解。这些技术中的一些最终可能会产生良好的结果，也许它们只需要一些调整来稳定训练过程。

5. 这一切意味着什么

YOLOv3是一款优秀的检测器。它速度快，精度高。在COCO数据集上以0.5至0.95交并比区间为标准的平均精度指标中表现不算顶尖，但在传统0.5交并比检测标准下表现非常出色。

我们为何要更换评估指标呢？最初的COCO论文中只有一句神秘的话：“评估服务器完成后将补充关于评估指标的完整讨论”。Russakovsky等人指出，人类很难区分0.3和0.5的IOU！“训练人类通过视觉检查来区分IOU为0.3和0.5的边界框非常困难——令人惊讶地困难。”[18] 如果人类都难以分辨差异，那这又有多重要呢？

但也许更好的问题是：“既然我们有了这些检测器，我们打算用它们来做什么？”从事这项研究的很多人都在谷歌和脸书工作。我想至少我们知道这项技术掌握在可靠的人手中，绝对不会被用来收集你的个人信息并卖给……等等，你是说这正是它将被用于的用途？？哦。

其他大力资助视觉研究的人就是军方，他们从未做过任何可怕的事情，比如用新技术杀死很多人哦等等……[1]

我抱有很大希望，大多数使用计算机视觉的人只是在用它做愉快的好事，比如统计国家公园里的斑马数量[13]，或者追踪在屋里闲逛的猫咪[19]。但计算机视觉已被投入可疑的用途，作为研究者，我们有责任至少思考我们的工作可能造成的危害，并设法减轻它。这是我们对世界应尽的责任。

最后，别@我。（因为我终于退出了推特）。

[1]The author is funded by the Office of Naval Research and Google.

References

[1] Analogy. *Wikipedia*, Mar 2018. 1

[2] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The pascal visual object classes (voc) challenge. *International journal of computer vision*, 88(2):303–338, 2010. 6

[3] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, and A. C. Berg. Dssd: Deconvolutional single shot detector. *arXiv preprint arXiv:1701.06659*, 2017. 3

[4] D. Gordon, A. Kembhavi, M. Rastegari, J. Redmon, D. Fox, and A. Farhadi. Iqa: Visual question answering in interactive environments. *arXiv preprint arXiv:1712.03316*, 2017. 1

[5] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 3

[6] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, et al. Speed/accuracy trade-offs for modern convolutional object detectors. 3

[7] I. Krasin, T. Duerig, N. Alldrin, V. Ferrari, S. Abu-El-Haija, A. Kuznetsova, H. Rom, J. Uijlings, S. Popov, A. Veit, S. Belongie, V. Gomes, A. Gupta, C. Sun, G. Chechik, D. Cai, Z. Feng, D. Narayanan, and K. Murphy. Open-images: A public dataset for large-scale multi-label and multi-class image classification. *Dataset available from https://github.com/openimages*, 2017. 2

[8] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2117–2125, 2017. 2, 3

[9] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*, 2017. 1, 3, 4

[10] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014. 2

[11] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016. 3

[12] I. Newton. *Philosophiae naturalis principia mathematica*. William Dawson & Sons Ltd., London, 1687. 1

[13] J. Parham, J. Crall, C. Stewart, T. Berger-Wolf, and D. Rubenstein. Animal population censusing at scale with citizen science and photographic identification. 2017. 4

[14] J. Redmon. Darknet: Open source neural networks in c. http://pjreddie.com/darknet/, 2013–2016. 3

[15] J. Redmon and A. Farhadi. Yolo9000: Better, faster, stronger. In *Computer Vision and Pattern Recognition (CVPR), 2017 IEEE Conference on*, pages 6517–6525. IEEE, 2017. 1, 2, 3

[16] J. Redmon and A. Farhadi. Yolov3: An incremental improvement. *arXiv*, 2018. 4

[17] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *arXiv preprint arXiv:1506.01497*, 2015. 2

[18] O. Russakovsky, L.-J. Li, and L. Fei-Fei. Best of both worlds: human-machine collaboration for object annotation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2121–2131, 2015. 4

[19] M. Scott. Smart camera gimbal bot scanlime:027, Dec 2017. 4

[20] A. Shrivastava, R. Sukthankar, J. Malik, and A. Gupta. Beyond skip connections: Top-down modulation for object detection. *arXiv preprint arXiv:1612.06851*, 2016. 3

[21] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. 2017. 3

参考文献

[1] 类比。*Wikipedia*，2018年3月。1[2] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, 和 A. Zisserman。PASCAL视觉对象类别（VOC）挑战赛。*International journal of computer vision*，88(2):303–338，2010年。6[3] C.-Y. Fu, W. Liu, A. Ranga, A. Tyagi, 和 A. C. Berg。DSSD：反卷积单次检测器。*arXiv preprint arXiv:1701.06659*，2017年。3[4] D. Gordon, A. Kembhavi, M. Rastegari, J. Redmon, D. Fox, 和 A. Farhadi。IQA：交互式环境中的视觉问答。*arXiv preprint arXiv:1712.03316*，2017年。1[5] K. He, X. Zhang, S. Ren, 和 J. Sun。用于图像识别的深度残差学习。收录于 *Proceedings of the IEEE conference on computer vision and pattern recognition*，第770–778页，2016年。3[6] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, 等。现代卷积目标检测器的速度/精度权衡。3[7] I. Krasin, T. Duerig, N. Alldrin, V. Ferrari, S. Abu-El-Haija, A. Kuznetsova, H. Rom, J. Uijlings, S. Popov, A. Veit, S. Belongie, V. Gomes, A. Gupta, C. Sun, G. Chechik, D. Cai, Z. Feng, D. Narayanan, 和 K. Murphy。OpenImages：一个用于大规模多标签和多类别图像分类的公共数据集。*Dataset available from https://github.com/openimages*，2017年。2[8] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, 和 S. Belongie。用于目标检测的特征金字塔网络。收录于 *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*，第2117–2125页，2017年。2, 3[9] T.-Y. Lin, P. Goyal, R. Girshick, K. He, 和 P. Dollár。用于密集目标检测的焦点损失。*arXiv preprint arXiv:1708.02002*，2017年。1, 3, 4[10] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, 和 C. L. Zitnick。Microsoft COCO：上下文中的常见物体。收录于 *European conference on computer vision*，第740–755页。Springer，2014年。2[11] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, 和 A. C. Berg。SSD：单次多框检测器。收录于 *European conference on computer vision*，第21–37页。Springer，2016年。3[12] I. Newton。*Philosophiae naturalis principia mathematica*。William Dawson & Sons Ltd.，伦敦，1687年。1[13] J. Parham, J. Crall, C. Stewart, T. Berger-Wolf, 和 D. Rubenstein。利用公民科学和照片识别进行大规模动物种群普查。2017年。4[14] J. Redmon。Darknet：C语言中的开源神经网络。http://pjreddie.com/darknet/，2013–2016年。3[15] J. Redmon 和 A. Farhadi。YOLO9000：更好、更快、更强。收录于 *Computer Vision and Pattern Recognition (CVPR), 2017 IEEE Conference on*，第6517–6525页。IEEE，2017年。1, 2, 3[16] J. Redmon 和 A. Farhadi。YOLOv3：一个渐进式的改进。*arXiv*，2018年。4[17] S. Ren, K. He, R. Girshick, 和 J. Sun。Faster R-CNN：利用区域提议网络实现实时目标检测。*arXiv preprint arXiv:1506.01497*，2015年。2[18] O. Russakovsky, L.-J. Li, 与 L. Fei-Fei。两全其美：人机协作进行物体标注。载于 *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*，第2121–2131页，2015年。4[19] M. Scott。智能相机云台机器人 scanlime:027，2017年12月。4[20] A. Shrivastava, R. Sukthankar, J. Malik, 与 A. Gupta。超越跳跃连接：用于物体检测的自顶向下调制。*arXiv preprint arXiv:1612.06851*，2016年。3[21] C. Szegedy, S. Ioffe, V. Vanhoucke, 与 A. A. Alemi。Inception-v4, Inception-ResNet 以及残差连接对学习的影响。2017年。3

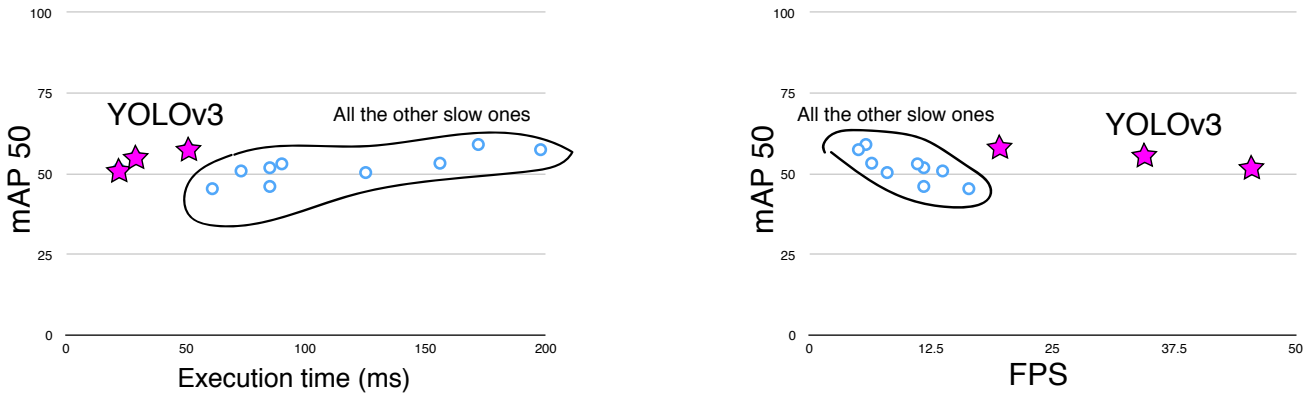

Figure 4. Zero-axis charts are probably more intellectually honest... and we can still screw with the variables to make ourselves look good!

Rebuttal

We would like to thank the Reddit commenters, labmates, emailers, and passing shouts in the hallway for their lovely, heartfelt words. If you, like me, are reviewing for ICCV then we know you probably have 37 other papers you could be reading that you'll invariably put off until the last week and then have some legend in the field email you about how you really should finish those reviews execept it won't entirely be clear what they're saying and maybe they're from the future? Anyway, this paper won't have become what it will in time be without all the work your past selves will have done also in the past but only a little bit further forward, not like all the way until now forward. And if you tweeted about it I wouldn't know. Just sayin.

Reviewer #2 AKA Dan Grossman (lol blinding who does that) insists that I point out here that our graphs have not one but two non-zero origins. You're absolutely right Dan, that's because it looks way better than admitting to ourselves that we're all just here battling over 2-3% mAP. But here are the requested graphs. I threw in one with FPS too because we look just like super good when we plot on FPS.

Reviewer #4 AKA JudasAdventus on Reddit writes "Entertaining read but the arguments against the MSCOCO metrics seem a bit weak". Well, I always knew you would be the one to turn on me Judas. You know how when you work on a project and it only comes out alright so you have to figure out some way to justify how what you did actually was pretty cool? I was basically trying to do that and I lashed out at the COCO metrics a little bit. But now that I've staked out this hill I may as well die on it.

See here's the thing, mAP is already sort of broken so an update to it should maybe address some of the issues with it or at least justify why the updated version is better in some way. And that's the big thing I took issue with was the lack of justification. For PASCAL VOC, the IOU threshold was "set deliberately low to account for inaccuracies in bounding boxes in the ground truth data" [2]. Does COCO have better labelling than VOC? This is definitely possible since COCO has segmentation masks maybe the labels are more trustworthy and thus we aren't as worried about inaccuracy. But again, my problem was the lack of justification.

The COCO metric emphasizes better bounding boxes but that emphasis must mean it de-emphasizes something else, in this case classification accuracy. Is there a good reason to think that more precise bounding boxes are more important than better classification? A miss-classified example is much more obvious than a bounding box that is slightly shifted.

mAP is already screwed up because all that matters is per-class rank ordering. For example, if your test set only has these two images then according to mAP two detectors that produce these results are JUST AS GOOD:

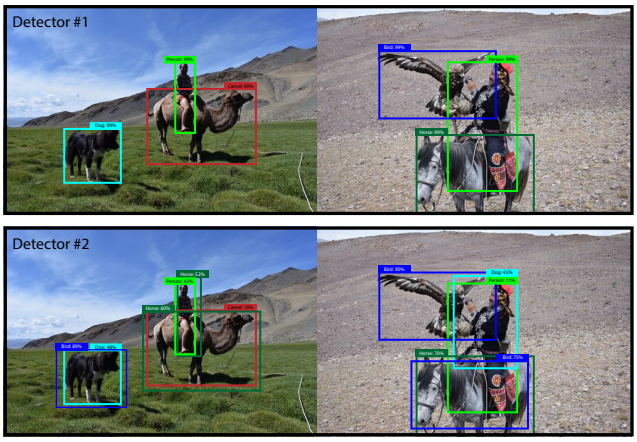

Figure 5. These two hypothetical detectors are perfect according to mAP over these two images. They are both perfect. Totally equal.

Now this is OBVIOUSLY an over-exaggeration of the problems with mAP but I guess my newly retconned point is that there are such obvious discrepancies between what people in the "real world" would care about and our current metrics that I think if we're going to come up with new metrics we should focus on these discrepancies. Also, like, it's already mean average precision, what do we even call the COCO metric, average mean average precision?

Here's a proposal, what people actually care about is given an image and a detector, how well will the detector find and classify objects in the image. What about getting rid of the per-class AP and just doing a global average precision? Or doing an AP calculation per-image and averaging over that?

Boxes are stupid anyway though, I'm probably a true believer in masks except I can't get YOLO to learn them.

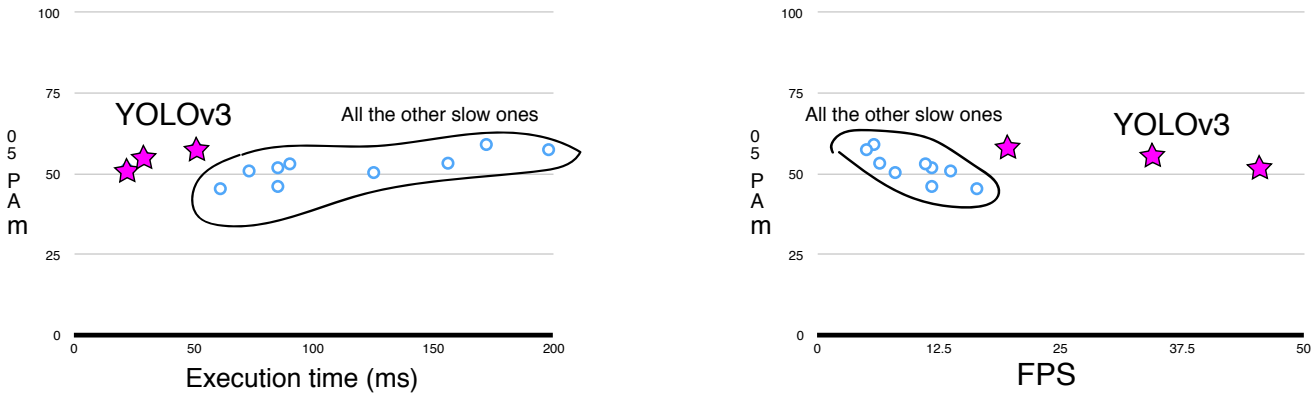

图4. 零轴图表可能在学术上更为诚实……我们仍可通过操纵变量让自己显得出色！

反驳

我们要感谢Reddit上的评论者、实验室的伙伴们、发来邮件的朋友们，以及在走廊里擦肩而过时喊出的那些温暖而真挚的话语。如果你和我一样，正在为ICCV审稿，那么我们猜你手头大概还有37篇论文等着读，而你总会拖到最后一刻，然后某位领域大牛会发邮件提醒你该完成那些审稿了——只是邮件内容可能有点让人摸不着头脑，说不定他们来自未来？总之，如果没有你们过去的自己（当然是在过去，只是稍微往前一点，不是一直到现在）付出的努力，这篇论文也不会成为它将来应有的样子。至于你有没有在推特上提过它，我可不会知道。就这么一说。

审稿人#2，又名Dan Grossman（哈哈，匿名审稿谁还这么干）坚持要我在此指出，我们的图表并非只有一个，而是有两个非零原点。你说得完全正确，Dan，那是因为这比承认我们所有人不过是在为2-3%的mAP争得面红耳赤要好看得多。但你要的图表在此奉上。我还额外加了一张带FPS的图，因为当我们以FPS为坐标绘图时，看起来简直棒极了。

审稿人#4，也就是Reddit上的JudasAdventus写道：“读起来挺有趣，但对MSCOCO指标的反对论点似乎有点站不住脚”。好吧，我早就知道你会是那个背叛我的人，犹大。你知道那种感觉吗？当你完成一个项目，结果只是差强人意，于是你得想方设法证明自己做的事其实挺酷的？我当时基本就在试图这么做，所以对COCO指标有点口不择言。但既然我已经占了这个山头，不如就战死在这里吧。

你看，问题在于mAP本身就已经有点问题了，所以它的更新版本或许应该解决它的一些问题，或者至少说明为什么更新版本在某些方面更好。而我最不满意的地方就是缺乏合理的解释。对于PASCAL VOC，IOU阈值“被故意设得较低，以考虑地面实况数据中边界框的不准确性”[2]。COCO的标注比VOC更好吗？这完全有可能，因为COCO有分割掩码，也许标签更可靠，因此我们不必那么担心不准确性。但再次强调，我的问题在于缺乏合理的解释。

COCO指标强调更好的边界框，但这种强调必然意味着它弱化了其他方面，在本例中是分类准确性。是否有充分理由认为更多精确的边界框比更好的分类更重要吗？一个错误分类的示例比一个略微偏移的边界框要明显得多。

mAP已经搞砸了，因为唯一重要的是每个类别的排序顺序。例如，如果你的测试集只有这两张图像，那么根据mAP，产生这些结果的两个检测器表现完全一样好：

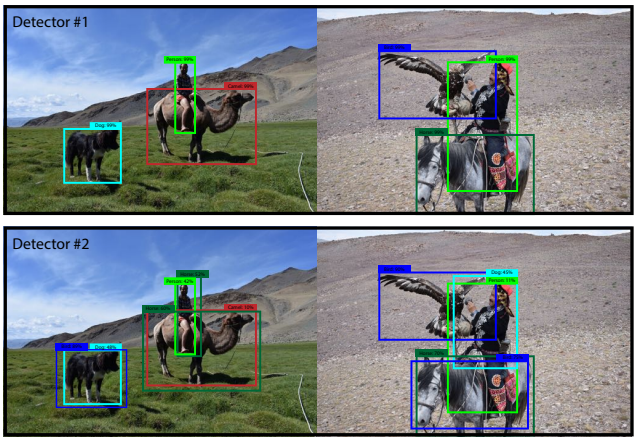

图5. 根据这两张图像的mAP指标，这两个假设检测器都表现完美。它们都完美无缺，完全等同。

这显然是对mAP问题的一种过度夸张，但我想我新近重新阐释的观点是：在“现实世界”中人们所关心的内容与我们当前的评估指标之间存在着如此明显的差异，我认为如果我们要制定新的指标，就应该聚焦于这些差异。再说了，这已经是平均精度均值（mean average precision）了，那我们到底该怎么称呼COCO指标呢，平均的平均精度均值（average mean average precision）吗？

这里有一个提议，人们真正关心的是：给定一张图像和一个检测器，该检测器在图像中查找和分类物体的表现如何。何不考虑取消每类AP（平均精度），只计算一个全局平均精度？或者对每张图像进行AP计算，然后取平均值？

不过盒子本来就很蠢，我可能是个真正的面具信徒，只是我没法让YOLO学会识别它们。